

15/12/2023

Zookeeper

Kafka

ZOOKEEPER

distributed

configuration management
system

ORCHESTRATOR

Coordination System

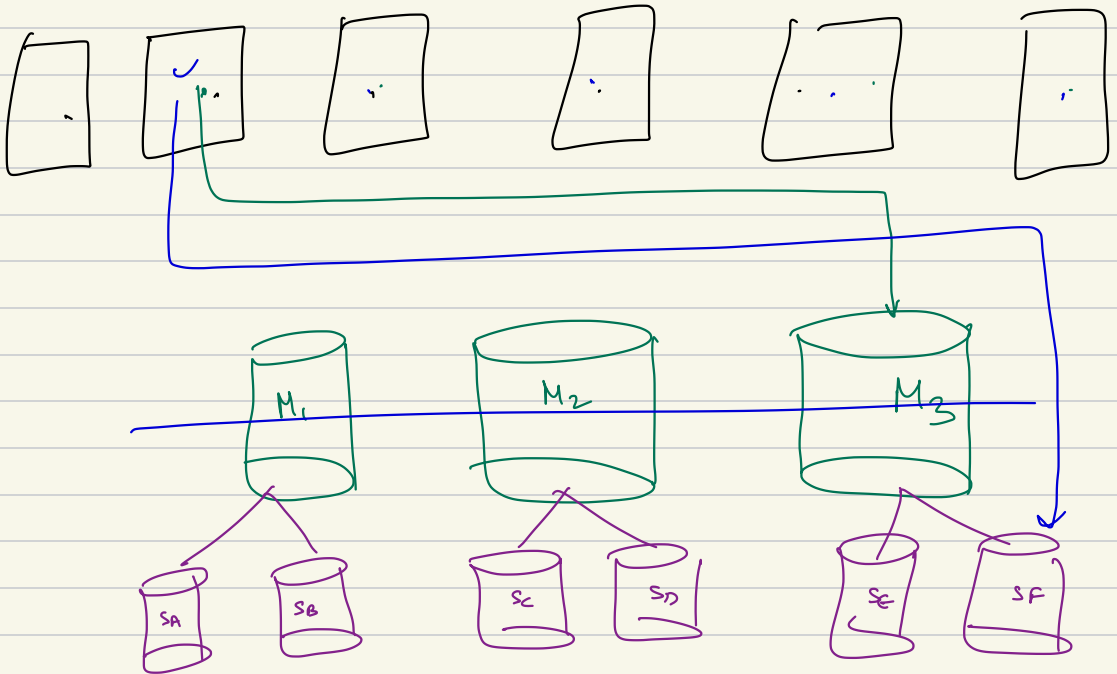
ip - addresses

Master/Slave

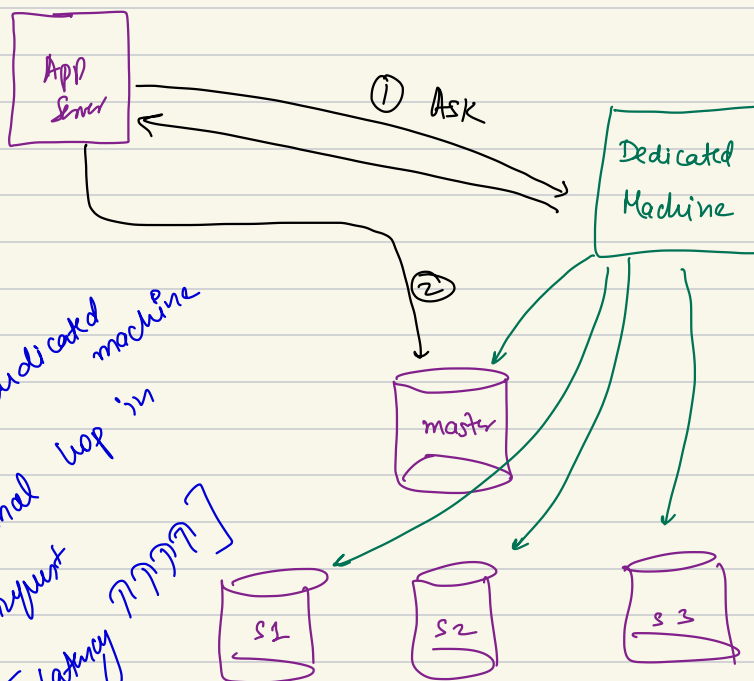
distributed locking

service discovery

Example problem:



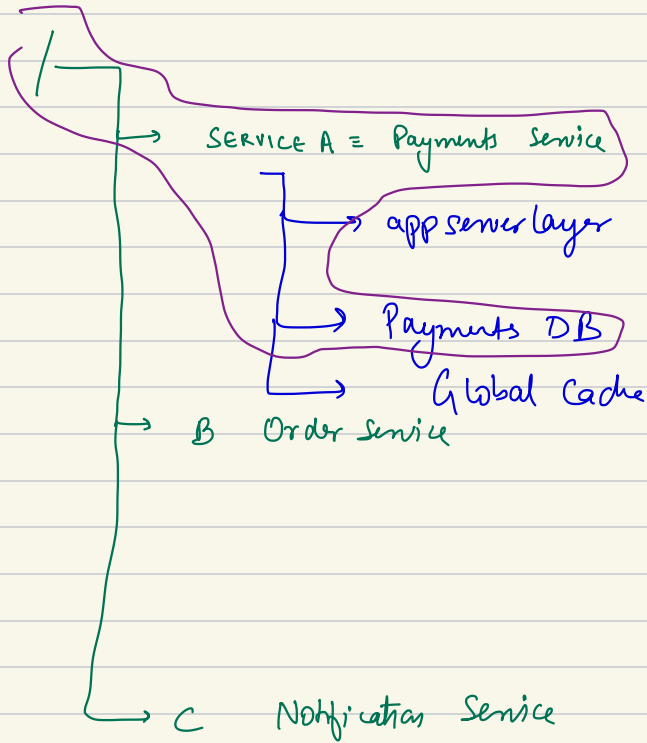
Naive Solution



- PROBLEMS
- ① SPOF of dedicated machine
 - ② Additional hop in every request [latency ↑↑↑↑]

Solution: Zookeeper 😊

zk is exactly file system



A | Payments DB

aws-key

sharding-config

shard 1

shard 2

master-ip

master-port

slave A-ip

slave A-port

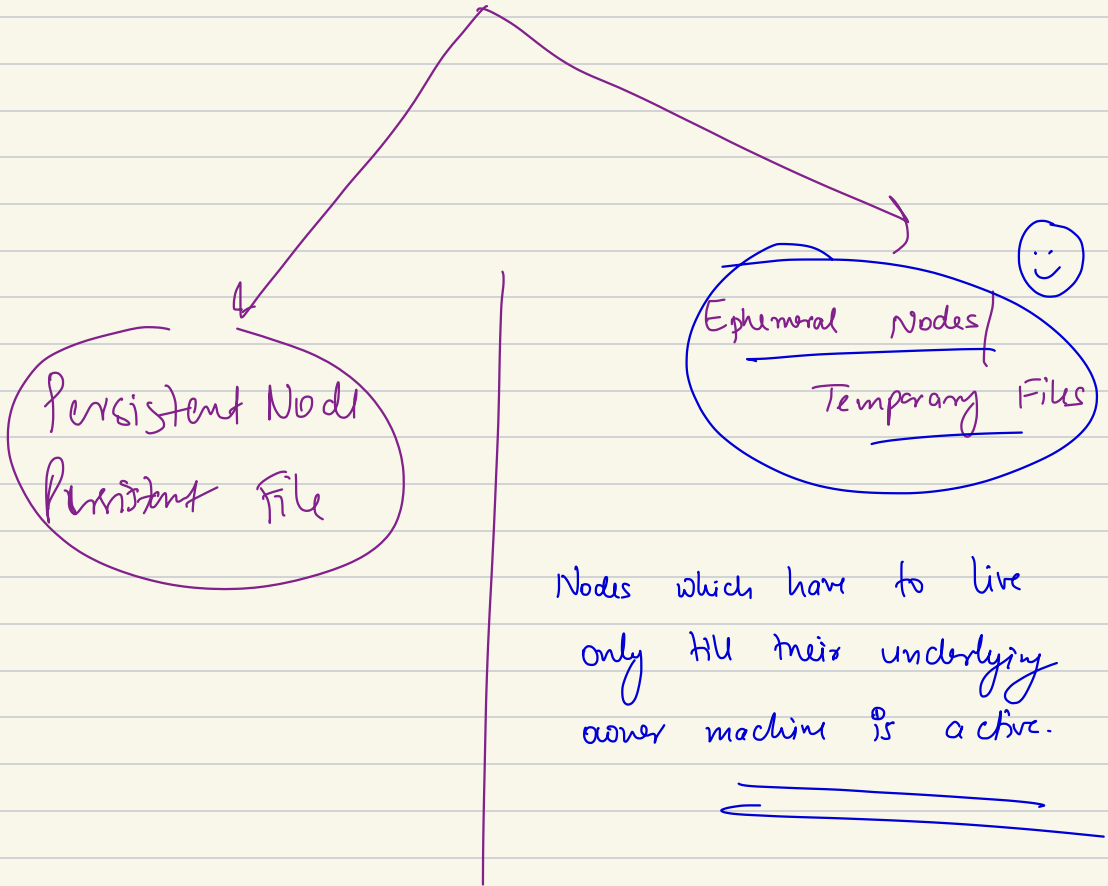
slave B-ip

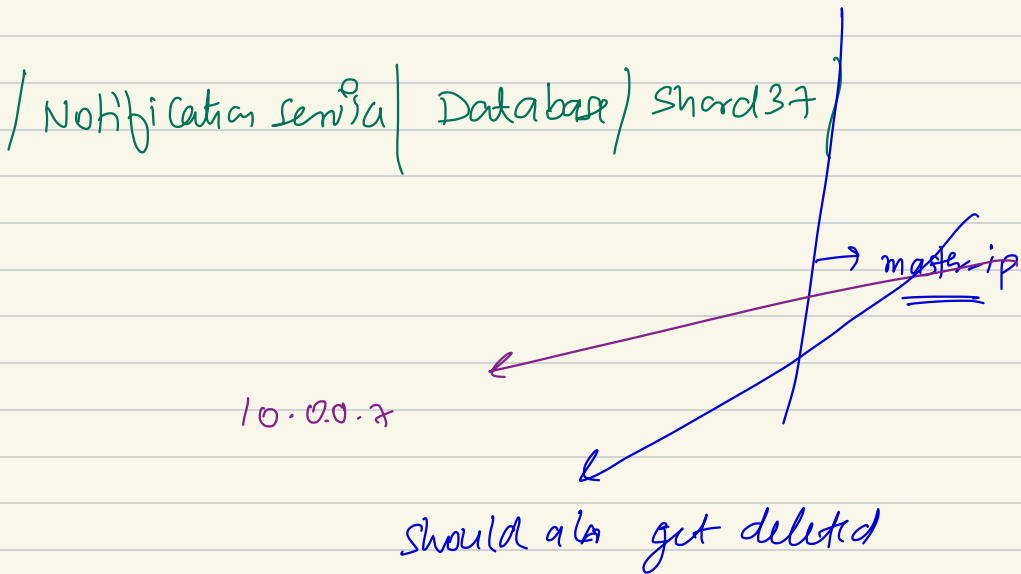
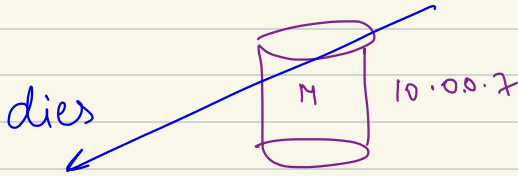
slave B-port

72.60.40.72

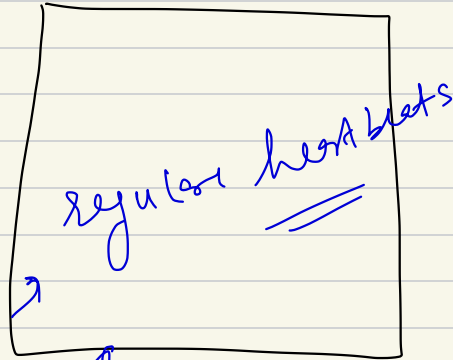
Green = Files
Purple = Folders

zkfile $\equiv$ zk Node 



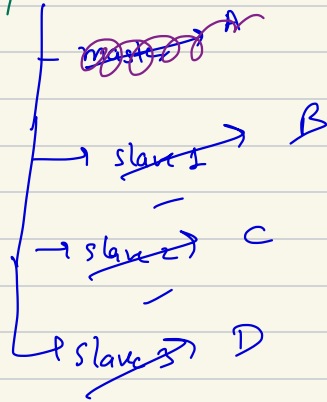


zk

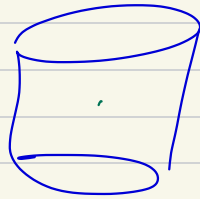
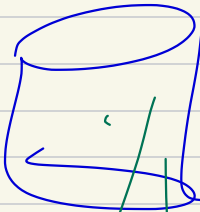
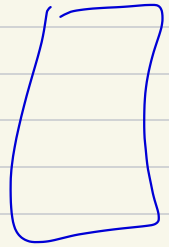
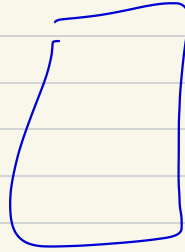
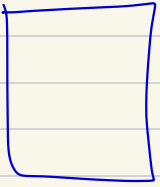


Notification service Database Shard 37

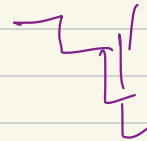
A, B, C, D, E IP Addresses



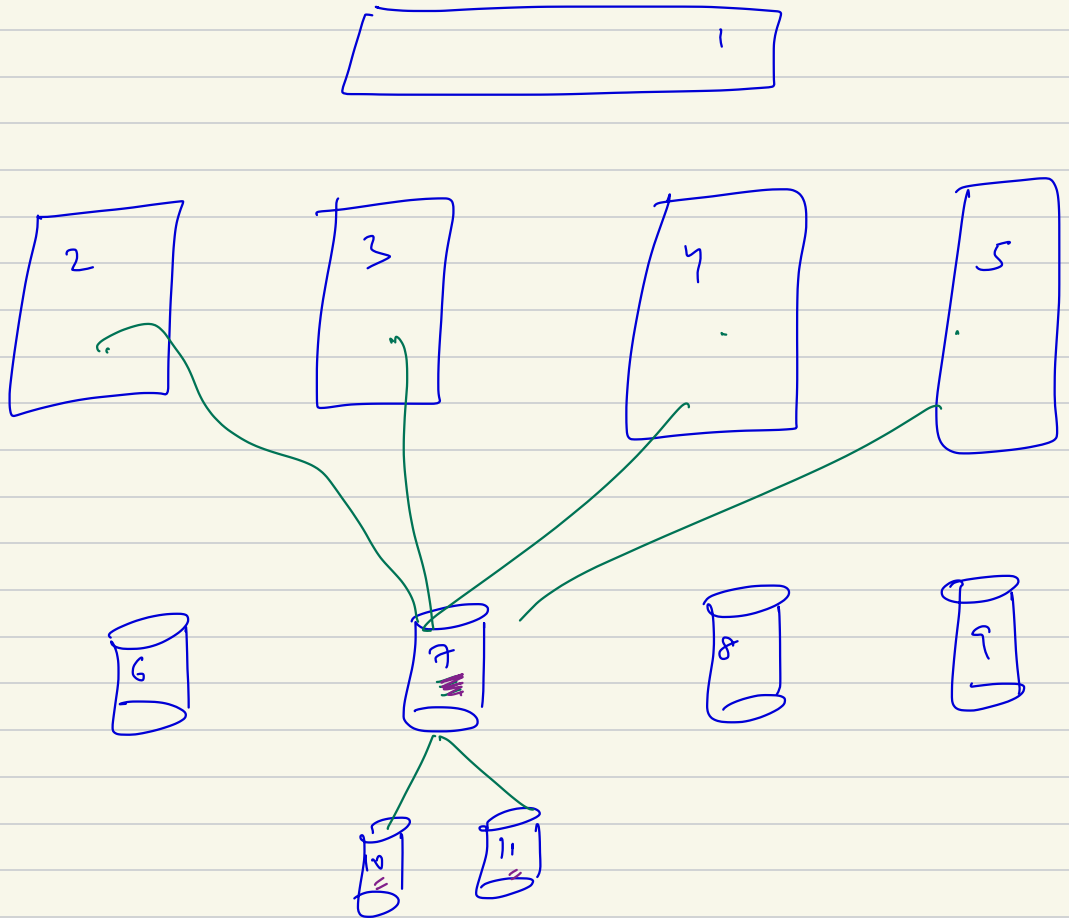
Gateway



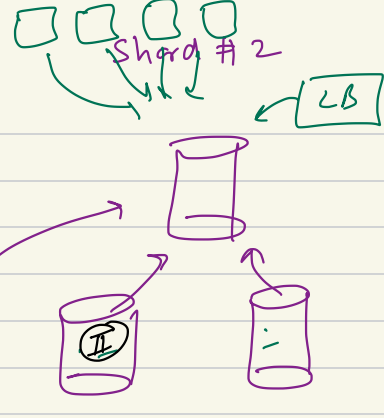
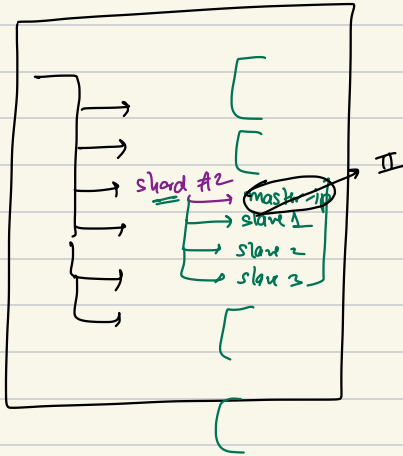
2K



zk watch



zk → File System



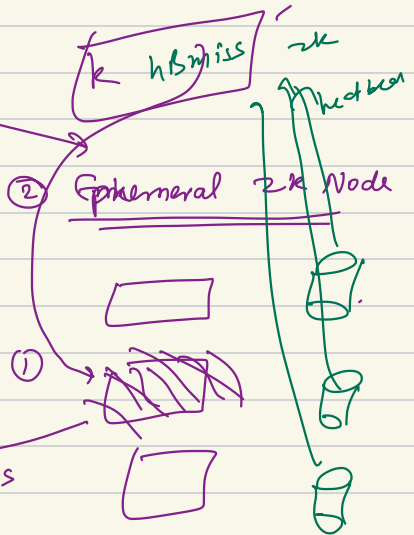
✓ zk ≡ File System

✓ zk Files ≡ zk Nodes

① Persistent zk Node

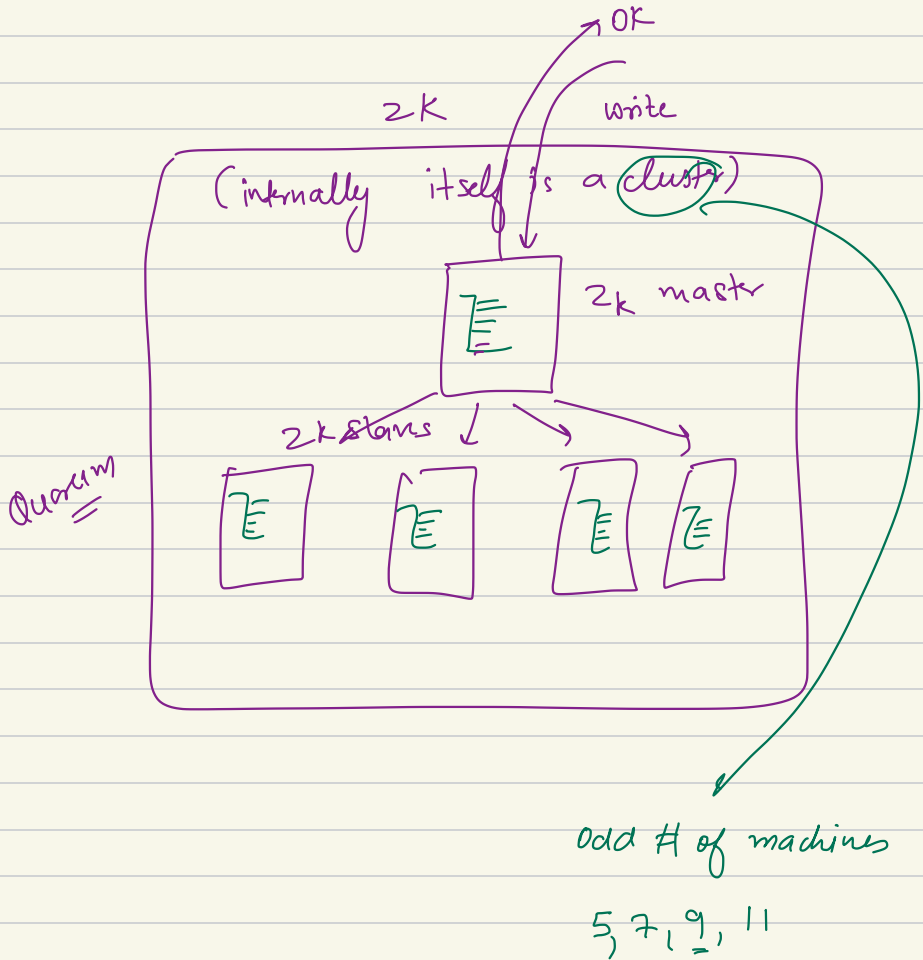
② Ephemeral zk Node

✓ ② notify all watchers

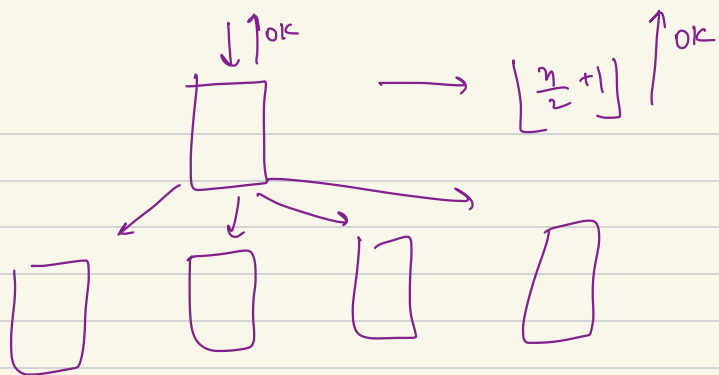


ID: 46 - Break

ID: 56 PM - Kafka



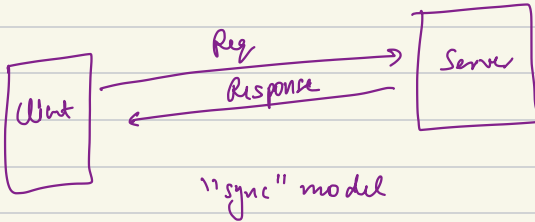
Quorum



10:48 - 10:58 PM ☺

ASync TASKS

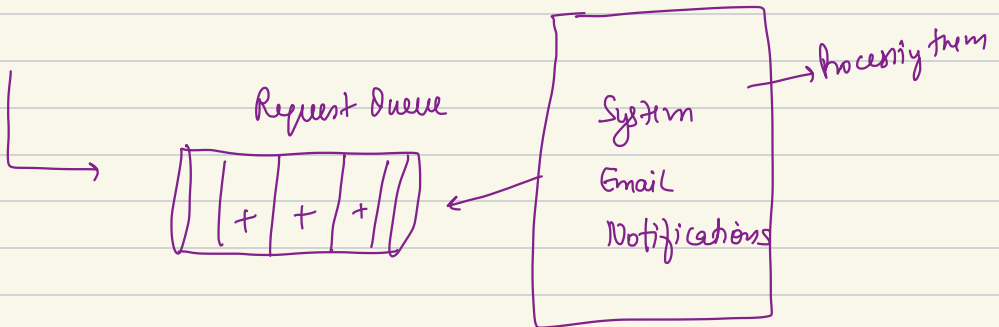
Request + Response Model



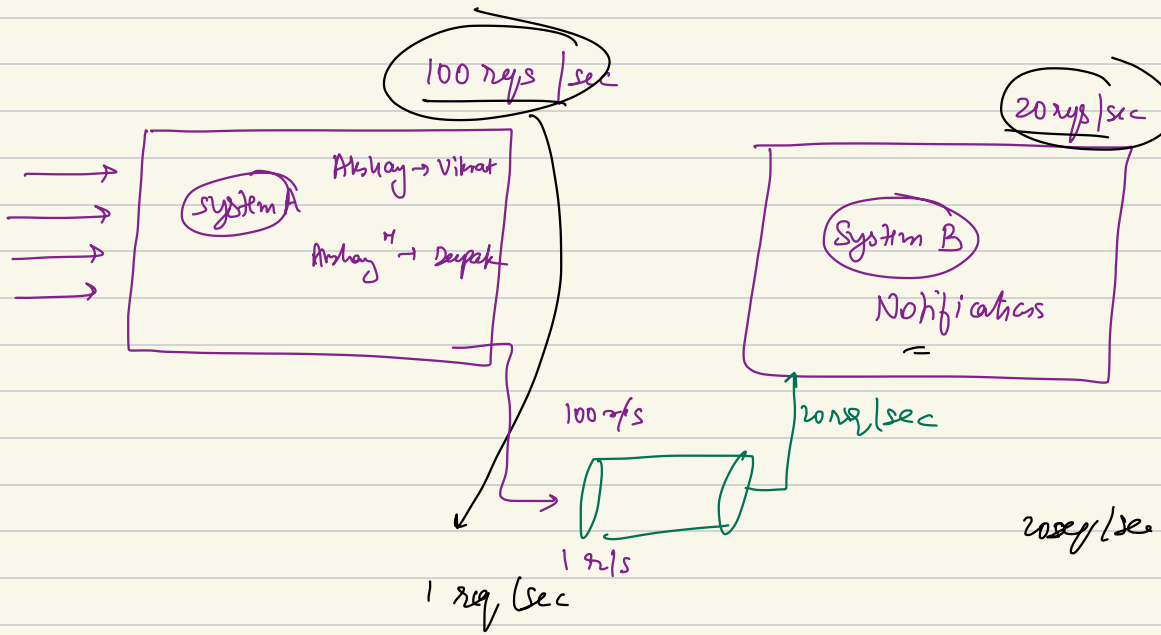
① Processing takes lot of time,
let's introduce a queue.

① non-blocking tasks

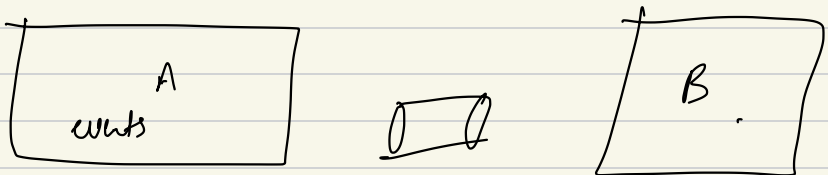
② Event Registered



✓ Notification Systems



② Rate of consumption $\neq$
Rate of production



③ If you want to do aggregation in b/w,

ASync TASKS → Persistent Queues

Publish Subscribe

ex.
Kafka ✓
RabbitMQ ✓

Pub-Sub

Message Queues

Kafka

① distributed ⇒ Horizontally Scalable

ex 10M msgs / second

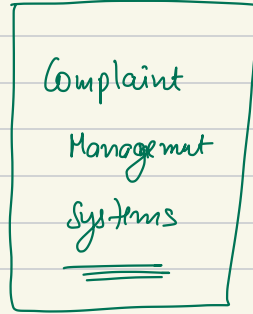
② Persistence ⇒ msgs are wrt to disk.

③ Realtime ⇒ Read Throughput and Latency is less

Flipkart

Users → Sending complaint emails

User Emails



Complaints / Requests to Sellers

Users
↓
Emails



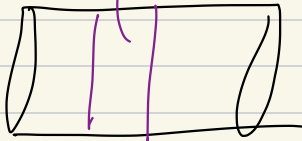
Producer

write



Publishing

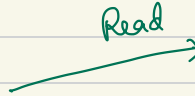
Producing



Kafka Queue

Event / Msg

Read



Subscribing

Consuming

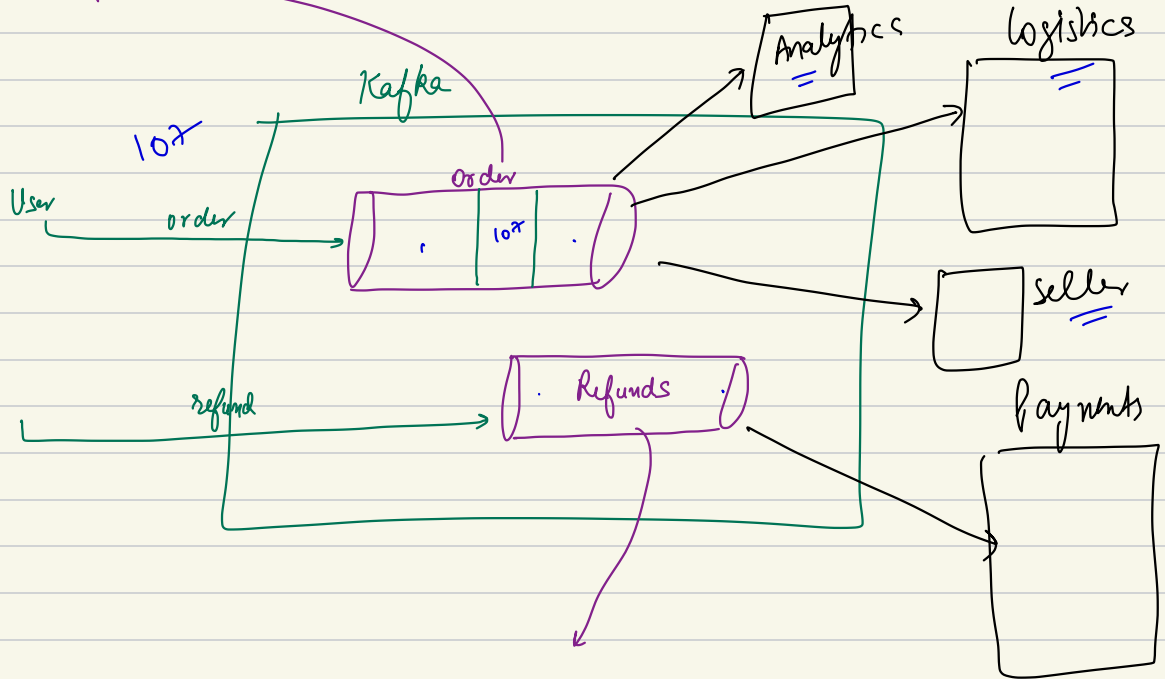


Consumer

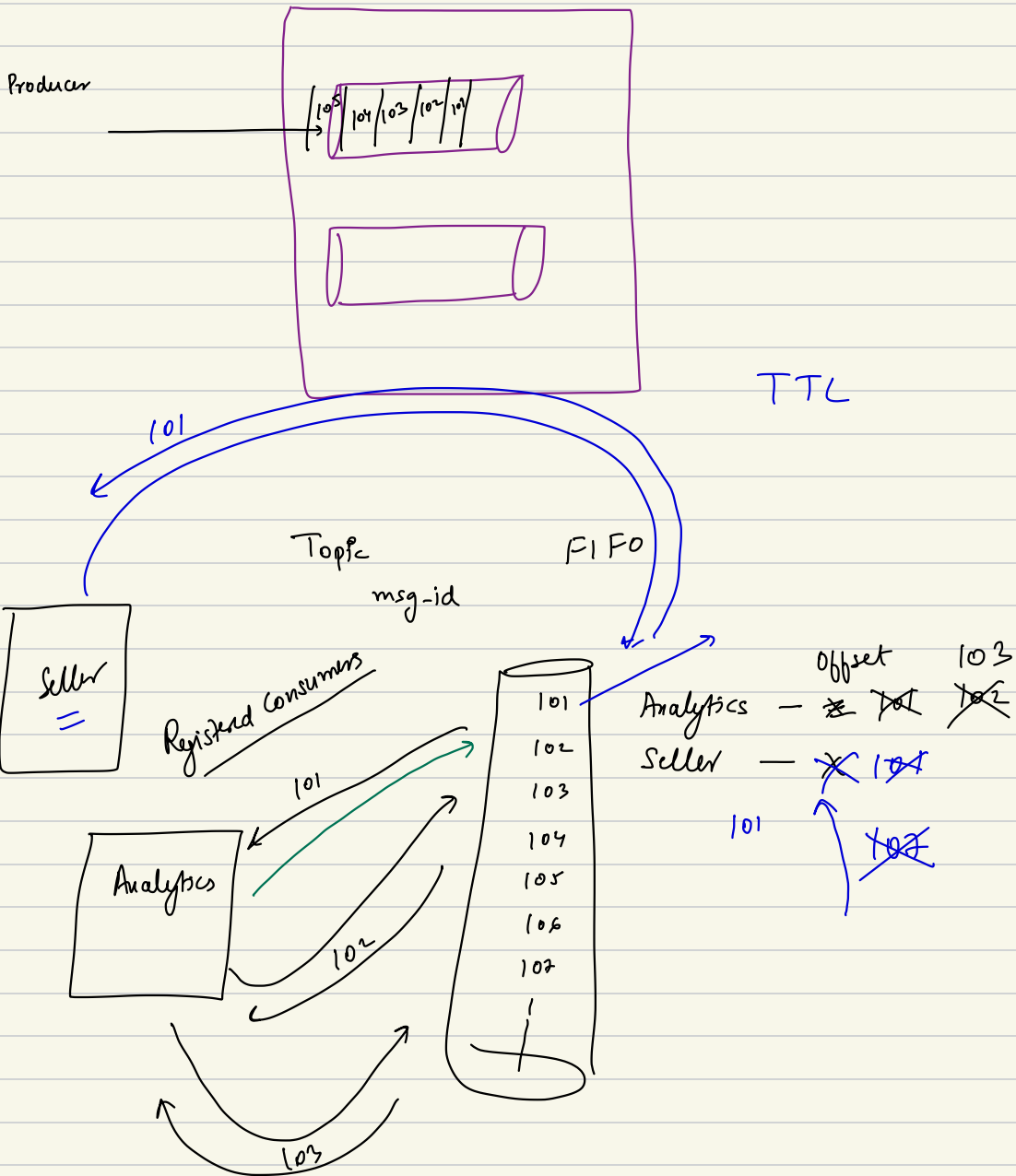
Kafka Queue / Message Queue

Collection of machines / cluster of machines

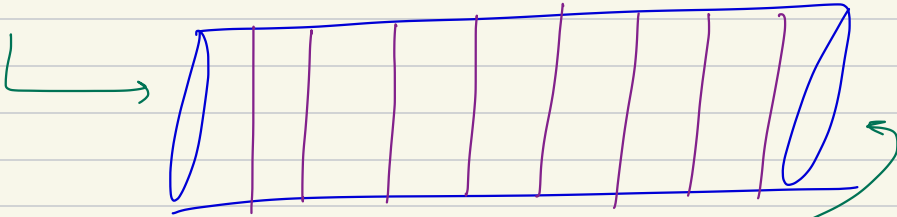
Kafka Topic



Kafka Topic



FIFO

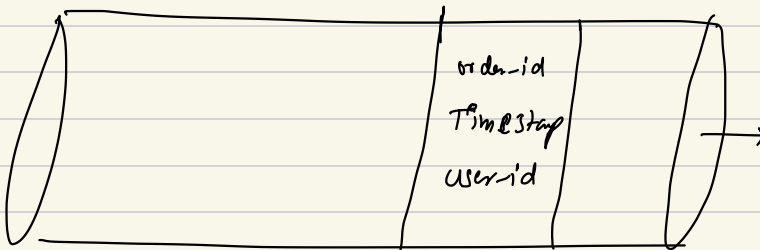


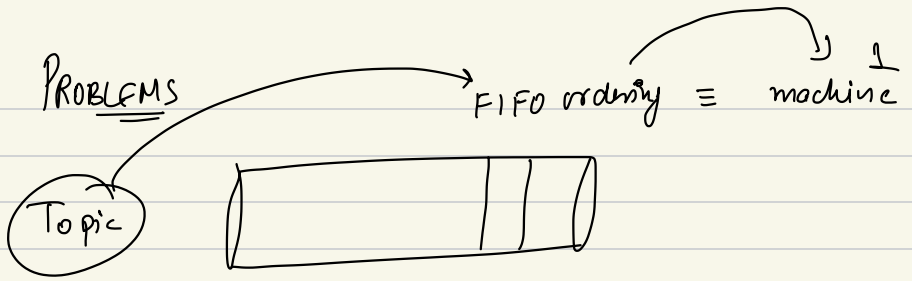
✓ Service A - ~~10~~ 102

✓ Service B - ~~10~~ 200 ~~800~~

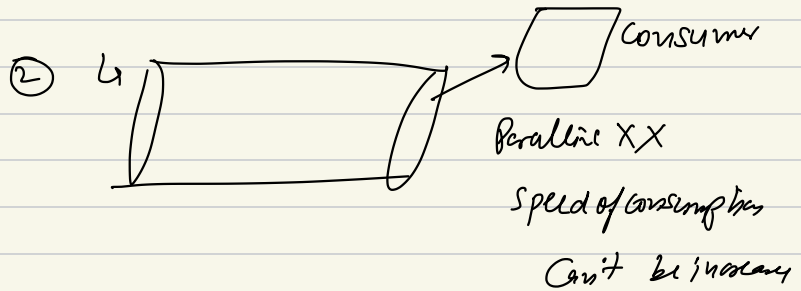
Might have a TTL which is independent of offsets.

Order queue





① I can't grow my queue beyond a point.

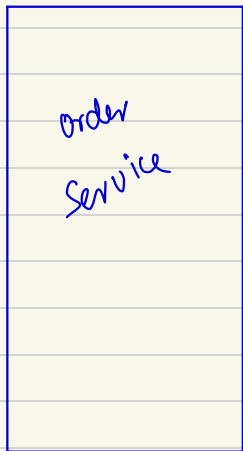


18/Dec/2023

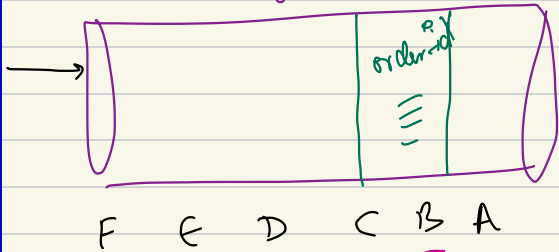
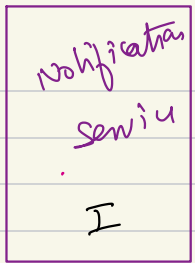
- ① Kafka — Partitions
- ~~②~~ Full Text Search
- ③ NLP Techniques
- ④ Elastic Search

swiggy





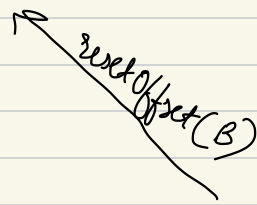
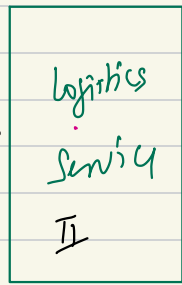
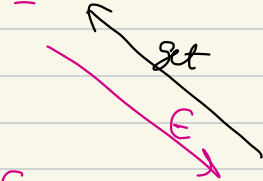
Kafka Topic = PlacedOrders



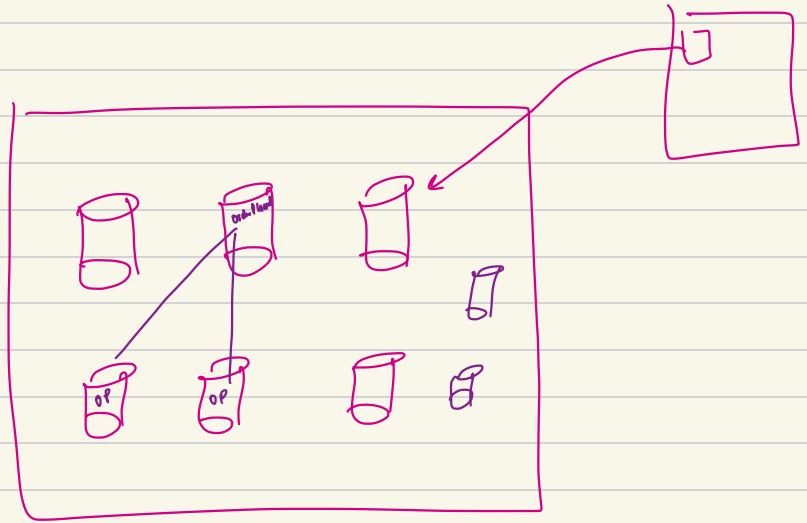
Producer

- I → C
- II → ~~D~~ E
- III → ~~C~~ B

offset



Consumers



Given that I want to build Kafka as a
dependable / fault tolerant system,

the fundamental concept of
replication should be employed 😊

There may be a situation that 1 entire machine may NOT be sufficient to store all msgs of 1 queue also.

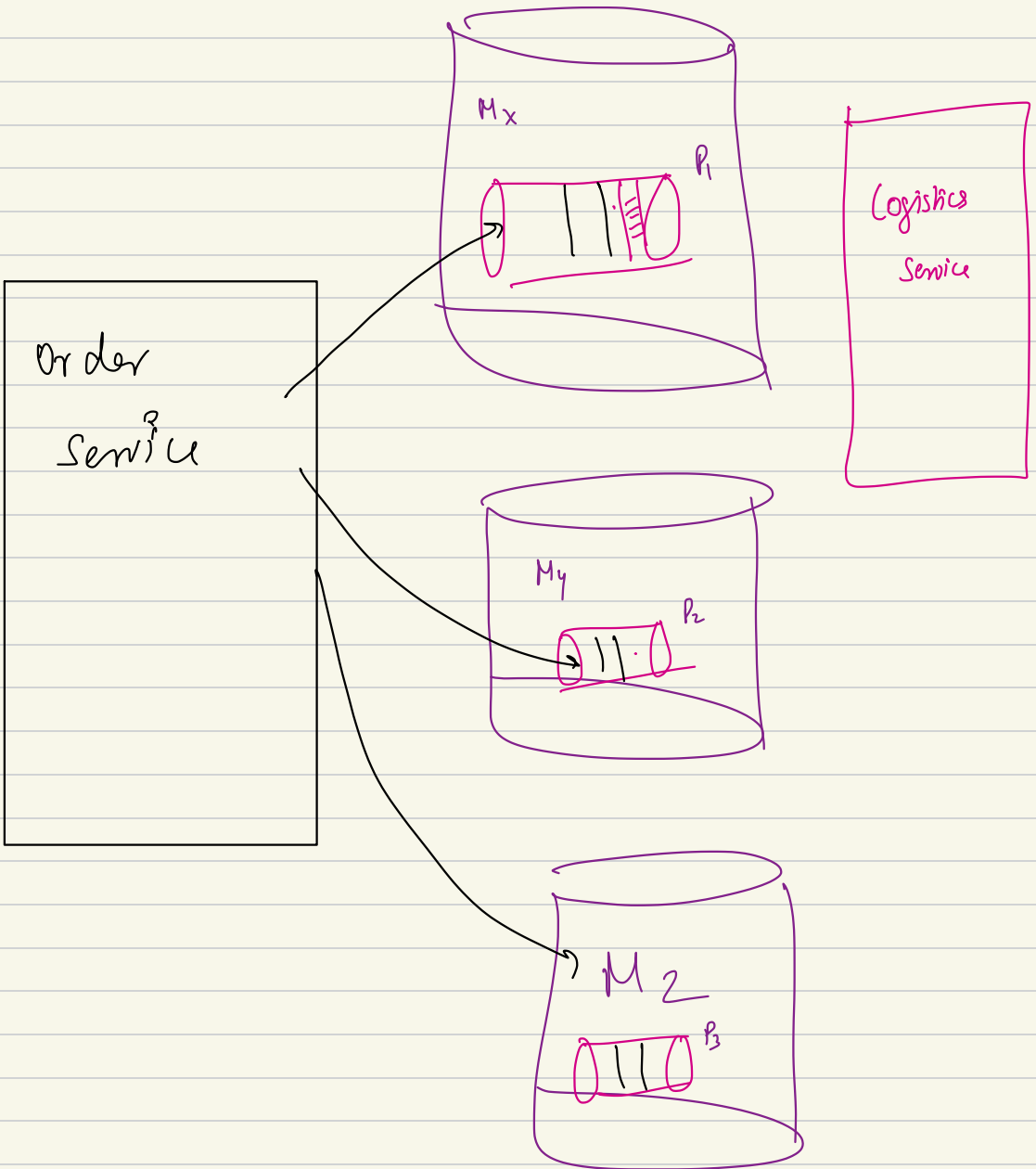
The usual way to solve this → SHARDING

But we can't use sharding directly before understanding it...

Queue is FIFO

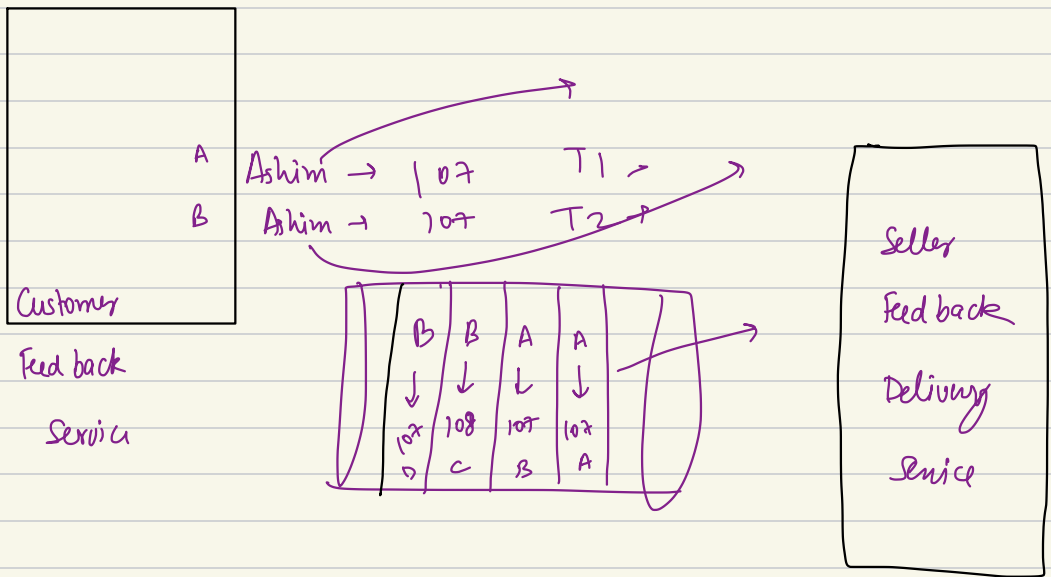
PROBLEM: when you divide a queue across multiple machines, it no longer remains 1 queue; it becomes multiple pieces of 1 queue.

Kafka Topic | Queue $\equiv$ Placed Orders



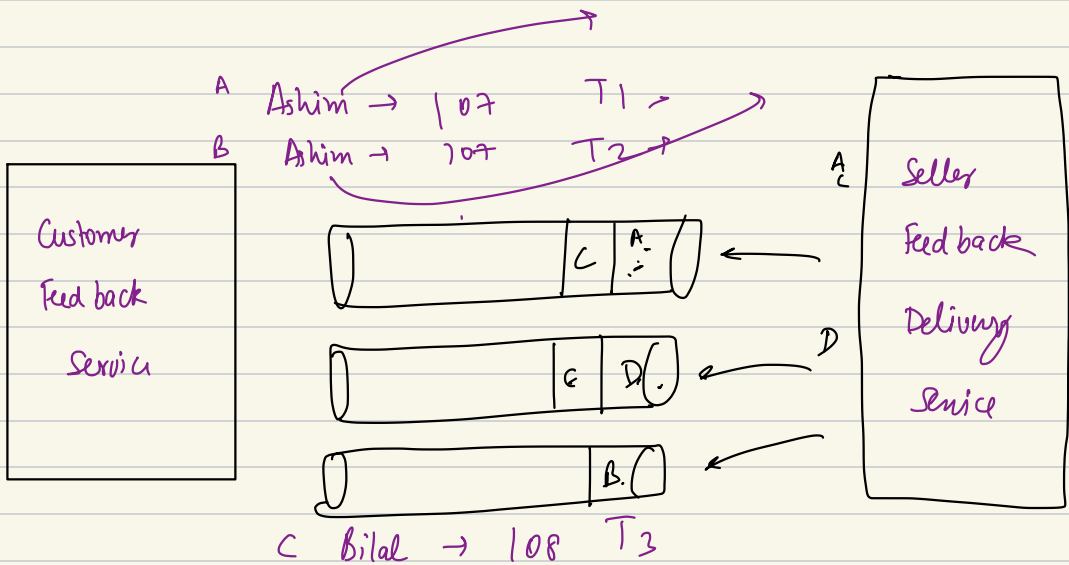
The Kafka Topic when partitioned, no longer gives FIFO guarantees across all partitions.

Only for msgs of the same partition, you can have FIFO guarantees.



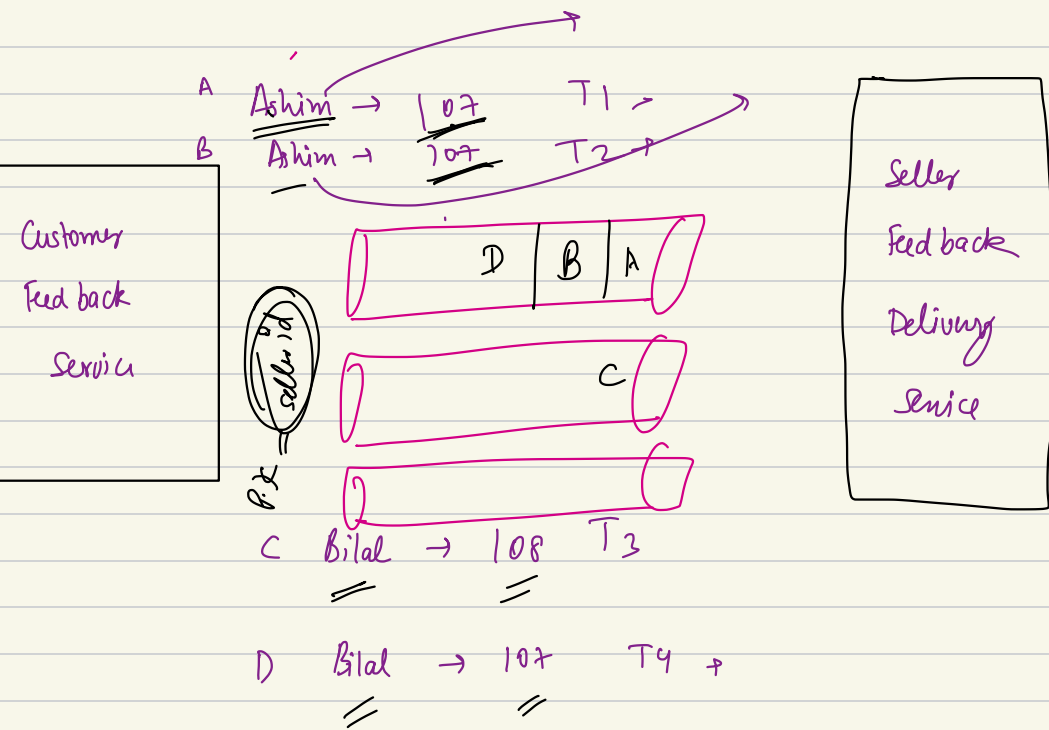
C Bilal → 108 T3

D Bilal → 107 T4



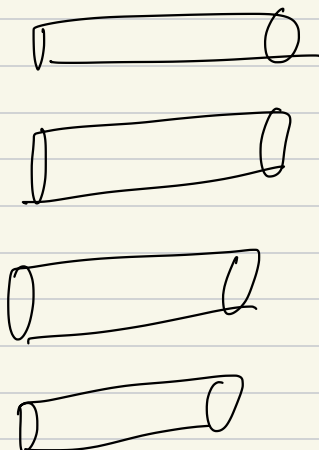
Order within the same partition is NOT
lost;
but global ordering is LOST

Kafka has to live with it

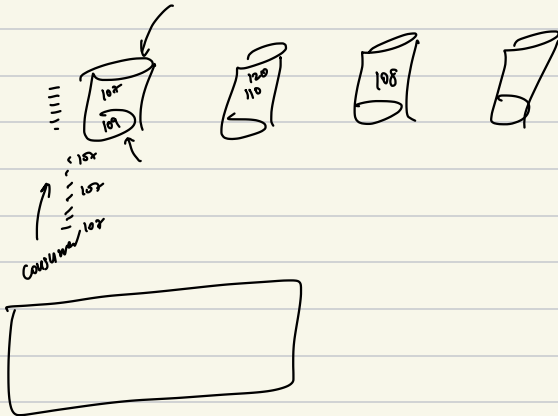


lp

Order



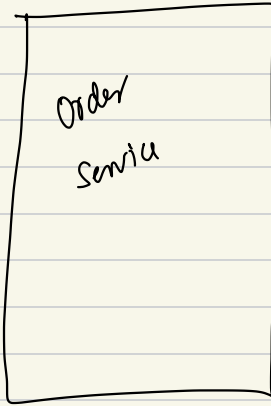
Restaurants



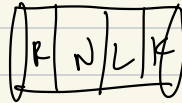
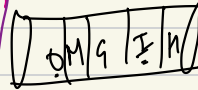
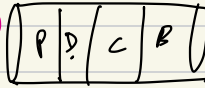
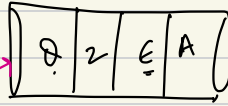
swiggy

Topic = Placed Orders

4 partitions



partition key



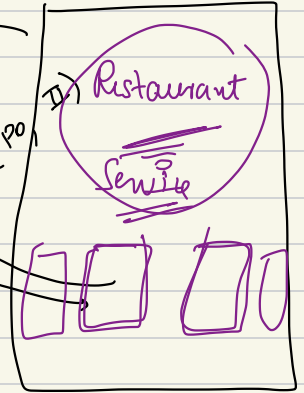
$poll(po, I)$

I

II

III

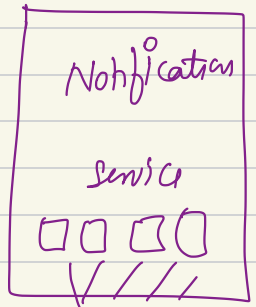
IV



Consumer group

R.S

I	→	D 2
II	→	P C
III	→	G
IV	→	N



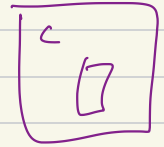
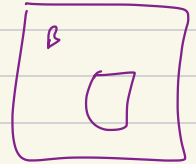
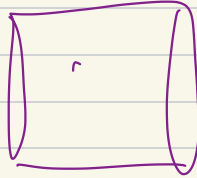
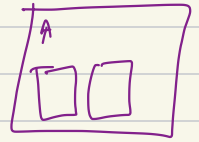
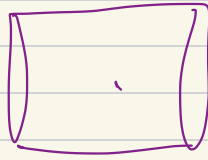
Consumer group

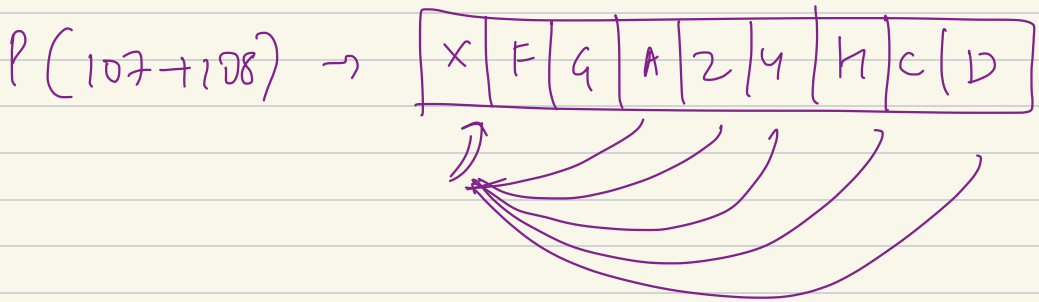
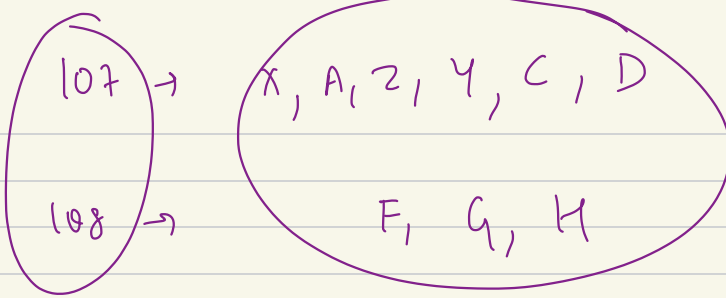
NS

I	-
II	-
III	-
IV	-

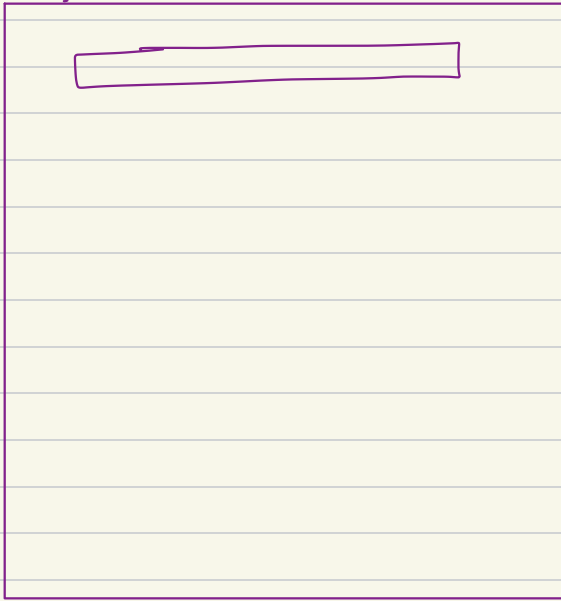
consume
5 services

Topic

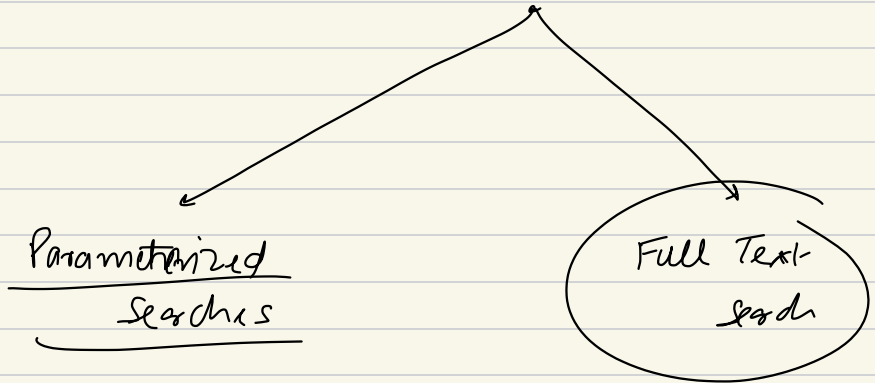




ELASTIC SEARCH



Linked In



A POST: India is a beautiful country

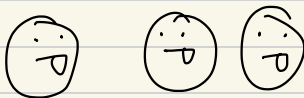
B POST : Canada India relationship is at a new low

C POST : Canada Engineering colleges might be a sham.

D POST: India lost the CWC

E POST : I love my job. I love my life.

query: (India cricket)

SOL 

Select * from CONTENT_TABLE

where content like '% india/cricket %'

NosQL

→ Document DB seems to be the most relevant DB out of all the DBs we studied to store about posts/web-pages

But still full text search is a challenge.
it is unoptimized to perform it.

FIRST PRINCIPLES $\Rightarrow$

Inverse Index

Glossary

Ram: 8, 10, 20, 24, 26

Aniket: 8, 80, 98, 101

Akash: 1, 100, 101

A Post:

India is a beautiful country

B Post:

Canada India relationship is at
a new low

C Post:

Canada engineering colleges might be
a sham.

D Post:

India lost the CWC

E Post:

I love my job. I love my life.

Inverted Index

India	-	A	B	D
Canada	-	B	C	
Country	-	A		
Beautiful	-	A		
Relationship	-	A		
<u>Cricket</u>	-	D		
<u>CWC</u>	-	D		

query:

"Cricket India"

Union
+
Ranking

How to create an inverted index.



Apache Lucene

① STOP WORD REMOVAL

a, an, the

② STEMMING

→ run, running, ran

→ crash, crashed, crashes

→ US of America, United States of America, USA

happy happiness happier → happy

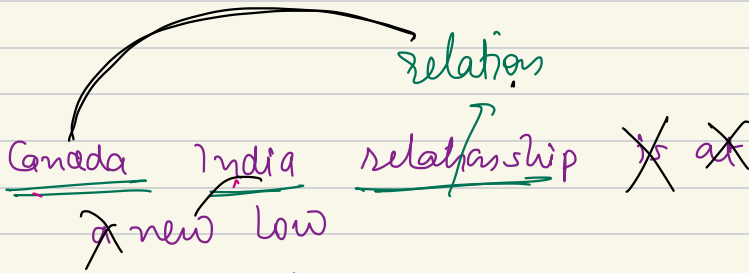
India, Indian, Indianess

③

③

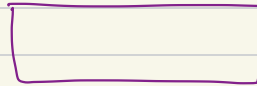
Tokenization

POST B: Canada India relationship is at
a new low



Input

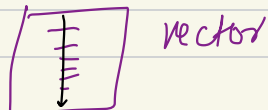
- Remove stopwords



stemming



Tokenization



vector

Unigram

Canada

India

relation

new

low

Bi-grams

Canada India

India relation

relation new

new low

Trigram

4-gram

5-gram

bi-skip-grams (1)

Tri-skip-grams (1)

Canada India relationship is at
a new low

after Tokenization

Canada

India

relation

new

low

Canada India

India relation

relation new

new low

Content



Vector of Tokens



Canada relation

India new

relation low

Linked In

1M posts

1 → Post ; Vector
2 → Post ; vector

3

4

1M → Post / Vector

All Tokens created globally



Assign some importance score
to each Token

Tf-IDf

